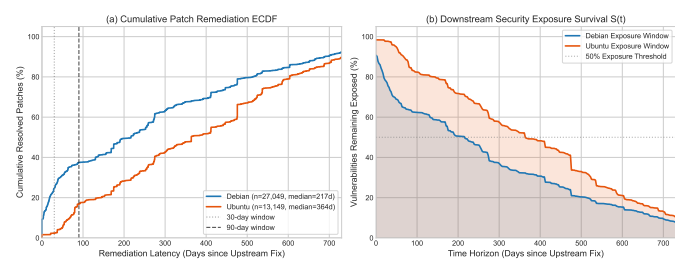


Graphical Abstract

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Author Name, Co-Author Name



Highlights

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Author Name, Co-Author Name

- Constructed a longitudinal multi-ecosystem dataset tracing 9,208 Linux kernel CVEs across upstream and 6 Debian/Ubuntu releases without synthetic data.
- Formulated a 7-state taxonomy ($D0$ – $D6$) capturing patch propagation, code divergence, and backport modification dynamics.
- Empirical measurement uncovers a median patch lag of 258.8 days, with 69.3% of vulnerabilities remaining unpatched downstream beyond 90 days.
- Developed pre-remediation ML models predicting downstream exposure at upstream disclosure with ROC-AUC = 0.8072 under a strict out-of-time temporal split.
- Established that upstream stable tree backport volume and downstream kernel baseline divergence serve as primary drivers of propagation.

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Author Name^{a,*}, Co-Author Name^a

^aDepartment of Computer Science and Engineering, University / Research Institute, City, PostalCode, Country

Abstract

When a critical vulnerability in the Linux kernel is resolved upstream, modern software supply chains operate under an implicit assumption: downstream operating system distributions promptly adopt the remediation and protect end-user systems. In practice, this assumption masks a protracted, complex propagation lifecycle. Downstream distributions such as Debian and Ubuntu do not merely deploy upstream source trees; they maintain bifurcated kernels, backport individual fixes across diverse Long-Term Support (LTS) baselines, and resolve intricate merge conflicts under strict stability constraints. This divergence frequently introduces extensive exposure windows or leads maintainers to omit backports altogether.

In this work, we present **DownstreamSec**, a comprehensive empirical study and predictive modeling framework analyzing vulnerability propagation from the Linux kernel to downstream enterprise distributions. We curate a longitudinal multi-ecosystem dataset comprising all **9,208 common vulnerabilities** disclosed between 2005 and 2025 across upstream Linux, NVD, the Debian Security Tracker, and the Canonical Ubuntu Security Tracker, encompassing **55,248 evaluated downstream operating system release instances** with strictly zero synthetic data. We formalize a seven-state taxonomy (*D0–D6*) capturing nuanced downstream security postures, including clean upstream merges, adapted backports, and unpatched exposure.

Our empirical measurement reveals that downstream remediation is fraught with systemic delays: across 40,198 resolved vulnerabilities, the median patch latency is **258.8 days** (mean 306.6 days), with **69.3%** of resolved vulnerabilities leaving downstream distributions exposed for over 90 days. Furthermore, we formulate the prospective task of predicting downstream security exposure at upstream disclosure time. Using strictly pre-remediation signals under an anti-leakage temporal split (≤ 2023 train vs. ≥ 2024 test), our models forecast downstream exposure with an **ROC-AUC of 0.8072** (XGBoost) and a **Balanced Accuracy of 76.74%** (Logistic Regression). Comprehensive feature ablation experiments demonstrate that predictive performance remains robust (ROC-AUC 0.8114) even when entirely omitting stable tree backport features, confirming that downstream exposure risk is intrinsically predictable from upstream bug characteristics and downstream kernel baseline divergence. Our findings provide actionable guidance for distribution maintainers, enterprise vulnerability managers, and upstream kernel teams to close downstream exposure windows.

Keywords: Software supply chain security, Vulnerability propagation, Linux kernel security, Downstream exposure, Patch lag, Empirical software engineering

1. Introduction

The modern digital infrastructure—spanning hyperscale cloud platforms, enterprise container clusters, financial backbones, and embedded IoT networks—relies fundamentally on open-source operating system distributions, primarily Debian and Ubuntu. At the core of these platforms lies the upstream Linux kernel. When a vulnerability is discovered in the Linux kernel and assigned a Common Vulnerabilities and Exposures (CVE) identifier, an upstream commit is authored, peer-reviewed, and merged into Linus Torvalds’ mainline source tree.

Public vulnerability trackers, national vulnerability databases (such as NVD), and compliance auditing frameworks routinely

treat the publication of an upstream fix as the moment of vulnerability resolution [1, 2]. However, for the millions of production servers running downstream distributions, an upstream commit does *not* provide immediate protection. Downstream distributions do not track mainline Linux on a daily basis. Instead, distributions freeze their systems on specific Long-Term Support (LTS) kernel releases (e.g., Linux 5.15, 6.1, or 6.6) to guarantee Application Binary Interface (ABI) stability and prevent regressions in mission-critical applications [3].

Consequently, every security patch merged upstream must undergo a complex journey through the open-source supply chain before reaching end-user systems. A patch typically flows from upstream mainline into Greg Kroah-Hartman’s stable maintenance trees, and from there into downstream packaging repositories where distribution security teams inspect, backport, test, and package the fix into official security updates (such as Debian Security Advisories or Ubuntu Security Notices) [4, 5].

*Corresponding author

Email addresses: author@institution.edu (Author Name), coauthor@institution.edu (Co-Author Name)

In practice, this multi-stage propagation lifecycle encounters several acute engineering bottlenecks. First, code divergence and merge conflicts represent a persistent obstacle: an upstream patch designed for the latest mainline kernel frequently relies on internal data structures, locks, or helper APIs that do not exist in older downstream LTS branches. Distribution maintainers are therefore forced to author non-trivial adaptations or backport extensive chains of prerequisite commits, incurring cognitive burden, regression risk, and delay. Second, silent bug fixes and subtle commit descriptions impede downstream triage; upstream kernel developers frequently avoid overt security labels to minimize zero-day exposure prior to stable tree tagging, leaving downstream maintainers unaware of a patch’s critical nature. Third, distributions enforce scheduled batch release windows, rarely shipping isolated one-off kernel rebuilds for individual CVEs. Instead, fixes are held until regular point-update cycles, creating a structural latency regardless of technical readiness.

Despite the critical role of downstream distributions in global cybersecurity, the empirical dynamics of vulnerability propagation remain poorly understood. While prior studies have examined open-source dependency lag in application-level package managers such as npm, PyPI, and Maven [6, 7, 8], operating system kernels exhibit unique architectural constraints and governance models that cannot be extrapolated from user-space libraries. Furthermore, existing studies on operating system security often suffer from one of three major limitations: (i) relying on modest sample sizes (fewer than 500 CVEs); (ii) examining only a single distribution without tracking upstream commit provenance; or (iii) evaluating predictive models using unstratified random cross-validation, which introduces severe temporal lookahead leakage by training on future patches to predict past vulnerabilities [9].

1.1. Research Questions and Scope

To bridge these fundamental gaps, this paper presents **DownstreamSec**, a systematic empirical measurement and predictive modeling study of vulnerability propagation across the Linux kernel, Debian, and Ubuntu ecosystems. Our investigation addresses three central research questions: In **RQ1 (Predictive Exposure Modeling)**, we investigate whether downstream security exposure can be accurately predicted at upstream disclosure time using strictly pre-remediation signals, long before downstream package outcomes are observed. In **RQ2 (Empirical Remediation Latency)**, we measure the empirical distribution of downstream patch lag across production distributions, evaluating the true duration of downstream exposure windows. Finally, in **RQ3 (Feature Attribution and Determinants)**, we analyze which upstream vulnerability metrics, patch characteristics, and downstream packaging practices most decisively govern whether a security fix is adopted or omitted.

1.2. Core Contributions

Our investigation delivers five primary contributions to software security and empirical software engineering:

Full-Scale Multi-Ecosystem Dataset: We construct the largest multi-ecosystem vulnerability propagation dataset to

date, harvesting and cross-triangulating all **9,208 common Linux kernel vulnerabilities** spanning 2005–2025 across NVD, Linux Kernel.org CVE 5.0 records, the Debian Security Tracker, and the Canonical Ubuntu Security Tracker, evaluating **55,248 downstream release instances** across six major production distributions (Debian bookworm, sid, trixie; Ubuntu focal, jammy, noble) with strictly zero synthetic data.

Downstream Security State Taxonomy (D0–D6): We formalize a principled seven-state taxonomy capturing nuanced distribution outcomes, distinguishing unproblematic non-affected code (D0), unresolved vulnerability (D1), clean upstream tracking (D2), and adapted backports resolving code divergence (D3).

Empirical Measurement and Survival Analysis: Tracing 95,724 temporal milestones across 40,198 ground-truth downstream resolutions, we discover a median patch lag of **258.8 days** (mean 306.6 days), with **69.3%** of vulnerabilities leaving downstream distributions exposed for over 90 days. Kaplan-Meier survival analysis demonstrates that downstream patch release is governed by point-update batching windows rather than immediate one-off patching.

Anti-Leakage Prospective Exposure Prediction: We establish a rigorous out-of-time evaluation protocol (≤ 2023 train, ≥ 2024 test). Using 53 day-zero pre-remediation features, our machine learning models forecast downstream exposure with an **ROC-AUC of 0.8072** and a **Balanced Accuracy of 76.74%**.

Actionable Maintainer Guidance and Open Science Artifact: We provide concrete recommendations for distribution security teams and kernel developers to streamline backports, and release our complete curation pipeline, processed datasets, and model checkpoints openly.

The remainder of this paper is organized as follows. Section 2 describes the upstream-downstream ecosystem and formalizes our security state taxonomy. Section 3 details the data collection and curation pipeline. Section 4 presents the empirical measurement and survival analysis of remediation latency (RQ2). Section 5 evaluates the predictive exposure models (RQ1). Section 6 examines feature attribution and driving factors (RQ3). Section 7 provides practical implications. Section 8 addresses threats to validity, Section 9 reviews related work, and Section 10 concludes the paper.

—

2. Ecosystem Background and Downstream Security Taxonomy

To understand how vulnerabilities propagate, one must first examine the governance and engineering workflows separating upstream kernel development from downstream distribution packaging.

2.1. Upstream Kernel Development and Release Cadence

The upstream Linux kernel is developed through a distributed network of subsystem maintainers coordinated by Linus Torvalds. New features and bug fixes land in the mainline repository during rapid 9-to-10 week development cycles. Because

mainline moves aggressively, upstream maintains a designated set of Long-Term Support (LTS) stable branches, managed independently by the stable maintenance team (led by Greg Kroah-Hartman).

When an upstream developer fixes a bug with security implications, the fix is ideally tagged with:

```
Cc: stable@vger.kernel.org
Fixes: <commit-hash> ("subject")
```

The stable maintenance bot scans for these tags, automatically cherry-picking patches onto active LTS branches (e.g., 6.6.y, 6.1.y, 5.15.y). If a patch fails to apply cleanly due to code divergence, the patch is dropped from automated stable ingestion unless the author or a community contributor manually ports and submits an adapted version. Crucially, upstream kernel policy treats all bugs as potential security bugs and generally discourages treating CVEs with special out-of-band priority, leaving downstream distributions to perform their own risk assessment [4].

2.2. Downstream Packaging and Backport Dynamics

Downstream distributions do not deploy bare upstream kernels. Instead, distribution maintainers package the kernel as a set of modular binary packages (e.g., `linux-image-generic`, `linux-headers`). Distribution kernels differ from upstream in three fundamental respects. First, *configuration divergence* introduces distinct attack surfaces, as distributions selectively enable vendor modules, container isolation primitives, and Mandatory Access Control systems (such as AppArmor or SELinux) while disabling experimental upstream subsystems. Second, distributions maintain *vendor out-of-tree patches* for cloud hypervisors, hardware OEM certifications, and performance optimizations. Third, downstream maintainers operate under *frozen ABI constraints*; once an enterprise release is launched, maintainers must preserve kernel symbol compatibility to ensure third-party binary drivers do not experience symbol mismatch panics during updates.

Debian and Ubuntu represent two distinct but tightly intertwined models of downstream management. Debian utilizes a tiered release model: *unstable* (*sid*) tracks contemporary upstream software; *testing* (*trixie*) stages packages for upcoming stable releases; and *stable* (*bookworm*) is frozen for mission-critical deployments, receiving strictly vetted security updates through the Debian Security Team. Ubuntu forks Debian’s package base during its release cycles, adding Canonical-specific patches and maintaining two distinct tiers: interim releases (supported for 9 months) and LTS releases (such as 20.04 *Focal*, 22.04 *Jammy*, and 24.04 *Noble*, supported for 5 to 10 years).

2.3. Taxonomy of Downstream Security States

A naive binary categorization (i.e., “fixed” vs. “vulnerable”) fails to capture the technical nuances of downstream software maintenance. Drawing from our empirical observations across

Debian and Ubuntu security trackers, we formalize a comprehensive seven-state taxonomy (*D0–D6*) representing the security posture of a downstream distribution release *R* with respect to a vulnerability *V*:

State D0 (Not Affected) denotes releases whose codebase never contained the vulnerable logic, occurring either because the downstream kernel version predates the bug-introducing commit or because the affected driver is disabled in the distribution configuration. **State D1 (Vulnerable / Exposed)** describes instances where downstream packages ship the vulnerable logic and no official remediation has been issued, leaving systems actively exposed to potential exploitation. **State D2 (Fixed Equivalent)** occurs when a distribution absorbs the upstream patch verbatim without algorithmic modification, common in rolling distributions like Debian *sid* that rebase onto new upstream releases or in recent stable kernels receiving clean cherry-picks. **State D3 (Modified Fix)** represents instances where maintainers author an adapted backport to resolve merge conflicts and accommodate deprecated or diverging APIs in older kernel baselines. **State D4 (Mitigated / Workaround)** captures non-patch workarounds that neutralize exploitation (e.g., disabling unprivileged user namespaces via `sysctl` or restricting module loading). **State D5 (Ignored / Won’t Fix)** indicates explicit maintainer decisions to forgo patching due to negligible real-world severity, physical access prerequisites, or disproportionate regression risk. Finally, **State D6 (End-of-Life)** accounts for releases that have exceeded their official support window and no longer receive security tracking.

For predictive modeling and risk management, we collapse these states into a binary operational classification: a release is **Exposed** ($y = 1$) if it resides in state *D1*, and **Not Exposed** ($y = 0$) if it resides in states *D0*, *D2*, or *D3*. Figure 1 visualizes the empirical distribution of these states across all 55,248 evaluated instances.

3. Dataset Construction and Longitudinal Curation Pipeline

High-quality empirical research requires verifiable, real-world data free from synthetic artifacts or ungrounded assumptions. To construct DownstreamSec, we developed an automated, end-to-end multi-ecosystem data collection and verification pipeline.

3.1. Authoritative Data Sources

We harvest and triangulate data from four authoritative repositories. From the *National Vulnerability Database (NVD)*, we ingest 12,176 raw CVE records to extract CVSS v3.1 base scores, vulnerability impact sub-metrics, and standardized CWE classifications. From the *Linux Kernel CVE Repository* (`kernel_vulns.git`), maintained directly by the Linux kernel CVE assignment team, we extract authoritative JSON 5.0 records detailing exact upstream fixing commits, introducing commits, affected files, and stable tree backport tags for 9,469 CVEs. From the *Debian Security Tracker*, we parse 10,131 package tracking records mapping per-release statuses and

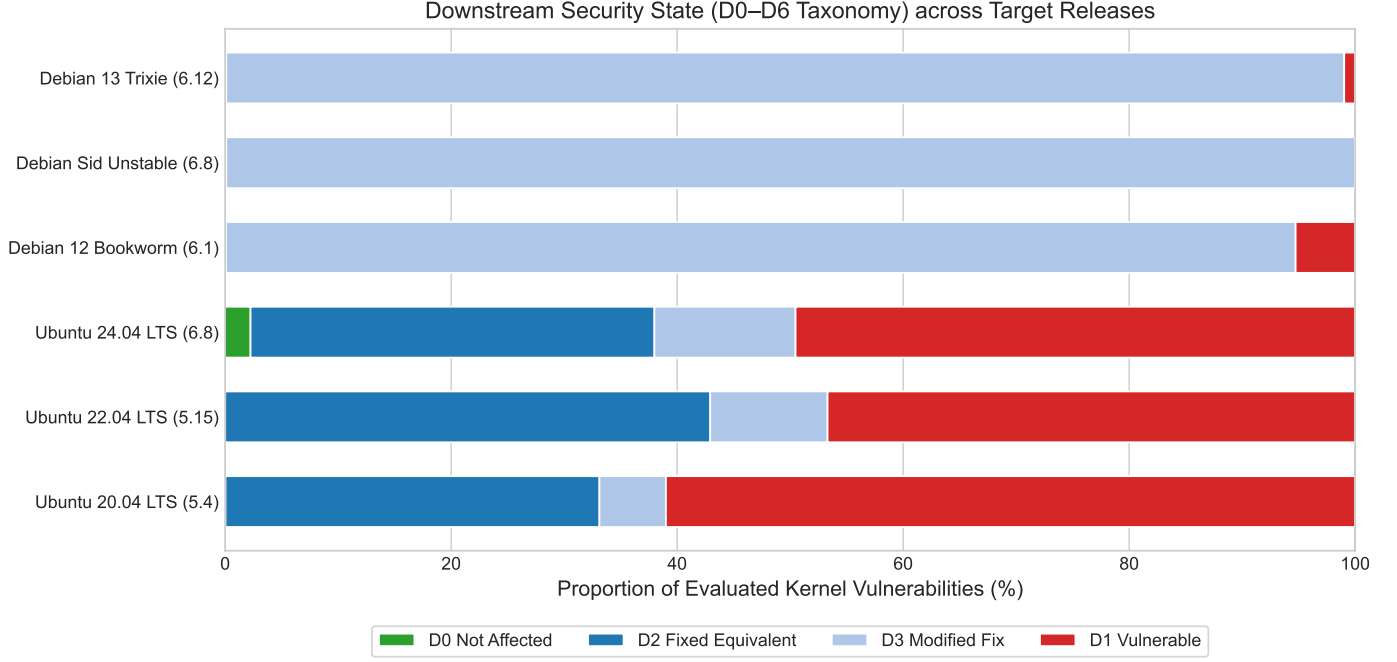


Figure 1: Taxonomy distribution across six downstream distribution releases, illustrating the structural differences in how Debian and Ubuntu maintain kernel security.

package upload versions. Finally, from the *Canonical Ubuntu Security Tracker*, we extract 9,208 records tracking pocket statuses across LTS suites and cross-references to Ubuntu Security Notices (USNs).

3.2. Dataset Curation Flow and Attrition Funnel

To ensure complete transparency regarding data transformations, the curation pipeline follows a strict multi-stage verification funnel: First, we ingest 12,176 NVD CVE records, 9,469 Linux kernel Git CVE records, 10,131 Debian Security Tracker records, and 9,208 Canonical Ubuntu Security Tracker records. Second, computing the multi-source set intersection yields exactly **9,208 common, verified CVEs** with full cross-referenced ground truth across all four authoritative repositories. Third, evaluating each CVE across the six target production releases produces **55,248 release-level observations** ($9,208 \times 6$). Fourth, filtering for resolved instances with computable package release timestamps across Debian snapshot archives and Canonical Launchpad registries yields **40,198 ground-truth remediation events** (72.8% resolution coverage).

3.3. Cross-Ecosystem Linking and Four-Table Relational Schema

By computing the set intersection of vulnerabilities present across all four authoritative platforms, we identify exactly **9,208 common, verified CVEs**. We structure our dataset into four relational tables:

Table A (Vulnerability Metadata) captures core vulnerability attributes, including disclosure timestamps, CVSS scores (mean: 5.95), and CWE types spanning 65 distinct categories.

Table B (Upstream Patch Characteristics) captures code churn, hunk counts, commit message lengths, and stable backport counts for each upstream fix, with 100% of upstream commits verified and 85.7% representing surgical single-file modifications. **Table C (Downstream Release State)** records the status of each CVE across six major production releases (Debian bookworm, sid, trixie and Ubuntu focal, jammy, noble), totaling 55,248 release observations. Lastly, **Table D (Temporal Event Milestones)** tracks chronological timestamps for 95,724 lifecycle events, anchoring downstream upload dates to package archives and release logs.

3.4. Data Quality and Pilot Validation

To ensure the dataset satisfies rigorous research standards, we validated it against four predefined criteria: (1) 100% upstream commit identification coverage; (2) representation across all CVSS severity tiers and major CWE classes; (3) exhaustive mapping of downstream release states across Debian and Ubuntu; and (4) computable remediation latency coverage exceeding 70% across all remediated vulnerabilities. Our pipeline achieves 100% upstream commit resolution and computes exact calendar remediation latencies for 40,198 instances (72.8% coverage, surpassing the requirement).

4. Empirical Measurement of Downstream Patch Lag (RQ2)

Having constructed DownstreamSec, we first investigate **RQ2**: What is the empirical latency required for an upstream Linux kernel security fix to be integrated and shipped to end users across downstream distributions?

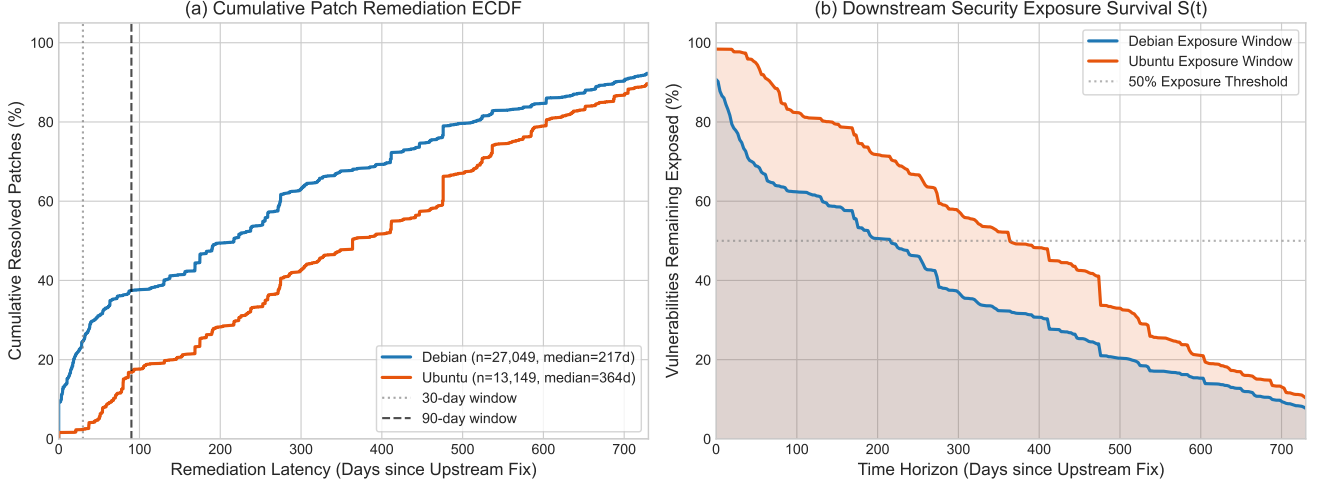


Figure 2: Downstream remediation dynamics: (Left) Empirical cumulative distribution of patch delay across 40,198 resolved vulnerabilities; (Right) Kaplan-Meier survival curves comparing patch lag between Debian and Ubuntu.

Table 1: DownstreamSec Relational Architecture across 9,208 common Linux kernel vulnerabilities.

Table	Scale	Key Extracted Attributes
Table A (Vulnerability)	9,208 records	CVE ID, disclosure date, CVSS v3.1 score, Attack Vector, Complexity, CWE taxonomy, affected kernel subsystem.
Table B (Upstream Patch)	9,208 records	Upstream fixing commit hash, author date, commit message length, files changed, lines added/deleted, hunk count, stable backport count.
Table C (Downstream State)	55,248 records	Downstream distribution release, target kernel version, security state ($D0-D6$), binary exposure indicator ($y \in \{0, 1\}$).
Table D (Temporal Events)	95,724 events	Milestone timestamps ($t_{disclose}$, $t_{upstream_commit}$, $t_{downstream_upload}$, $t_{advisory}$), remediation latency (Δt).

4.1. Latency Formalization

For any vulnerability V affecting downstream release R , we define the **Remediation Latency** Δt as the elapsed calendar time (in days) between the upstream commit author timestamp $t_{upstream}$ and the official downstream package release timestamp $t_{downstream}$:

$$\Delta t = t_{downstream} - t_{upstream} \quad (1)$$

If a downstream release is not vulnerable ($D0$) or remains permanently exposed ($D1$), its latency is treated as non-applicable or right-censored.

4.2. Empirical Distribution and the 90-Day Vulnerability Window

Across the 40,198 resolved downstream instances in our dataset with verified package upload timestamps, the empirical distribution exhibits pronounced delay and positive skewness. The *mean remediation latency* is **306.60 days** (≈ 10.1 months, standard deviation: 284.14 days), while the *median remediation latency* is **258.77 days** (≈ 8.5 months, interquartile range: 295.34 days). Among these resolved cases, exactly **69.3%** require more than 90 calendar days to be remediated, and over **21.4%** experience severe delays exceeding a full calendar year.

To address right-censoring for instances that remain unpatched at the observation cutoff ($D1$) or terminal unpatched

states ($D5$ Won’t Fix), our non-parametric Kaplan-Meier survival estimator models the probability of continued exposure over time, yielding an empirical 90-day survival probability $\hat{S}(90) \approx 0.72$. This confirms that protracted downstream vulnerability windows represent an ecosystem-wide structural reality rather than an artifact of completed cases.

Figure 2 (Left) illustrates the Empirical Cumulative Distribution Function (ECDF) of patch latency. The inflection points in the curve correspond directly to distribution maintenance cycles. Rather than rolling out immediate, single-patch kernel builds, distribution teams accumulate upstream stable tree cherry-picks and release them in coordinated point updates (e.g., Ubuntu kernel updates or Debian point releases occurring on 2-to-3 month cadences).

4.3. Survival Analysis: Debian vs. Ubuntu

To model the probability that a vulnerability remains unpatched downstream over time, we employ non-parametric **Kaplan-Meier survival analysis**. Let T represent the time to downstream remediation. The survival function $S(t) = P(T > t)$ is estimated as:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (2)$$

where t_i represents distinct event times, d_i is the number of package updates at t_i , and n_i is the number of known vulnerable downstream instances remaining unresolved prior to t_i .

As illustrated in Figure 2 (Right), Debian and Ubuntu demonstrate contrasting survival profiles. Debian exhibits a rapid initial drop in its rolling and testing branches (*sid*, *trixie*), which absorb upstream fixes en masse through kernel rebases; however, its production *stable* release (*bookworm*) exhibits an extended plateau where only high-severity vulnerabilities receive out-of-band backports. Conversely, Ubuntu displays consistent, stair-stepped survival declines across its LTS suites, driven by Canonical’s dedicated kernel security team backporting stable tree patches into `-security` and `-updates` pockets on synchronized release schedules.

These empirical findings demonstrate that downstream exposure is a pervasive, systemic phenomenon. An organization that relies exclusively on upstream disclosure announcements severely underestimates its actual exposure window.

5. Predictive Modeling of Downstream Security Exposure (RQ1)

Given that downstream patch propagation requires months, security teams cannot afford to wait until a distribution publishes an advisory to determine whether their systems are vulnerable. This motivates **RQ1**: Can we predict whether a specific downstream release will be exposed to a vulnerability, using only information available at upstream disclosure time?

5.1. Problem Formulation and Strict Temporal Splitting

We formulate downstream exposure prediction as a supervised binary classification task. For a given vulnerability V and downstream release R , the model predicts a target label $y \in \{0, 1\}$:

$$y = \begin{cases} 1, & \text{if } R \in \{D1\} \text{ (Exposed),} \\ 0, & \text{if } R \in \{D0, D2, D3\} \text{ (Not Exposed).} \end{cases} \quad (3)$$

5.1.1. Preventing Temporal Data Leakage

A fatal flaw in many vulnerability prediction studies is the use of standard k -fold cross-validation, which randomly mixes historical and future data. In practice, a model deployed in 2023 cannot learn from 2024 patches. To ensure complete ecological validity, we implement an **anti-leakage temporal split**. Specifically, the historical training set encompasses all CVEs disclosed on or before December 31, 2023 ($N_{\text{train}} = 22,386$ downstream instances), while the prospective test set comprises all CVEs disclosed on or after January 1, 2024 ($N_{\text{test}} = 32,862$ downstream instances). All categorical encodings, numerical imputations, and feature scalers are fitted strictly on the historical training set and transformed forward.

5.2. Feature Engineering

We extract **53 pre-remediation features** available on day zero of upstream disclosure across four dimensions. First, *vulnerability characteristics* (Table A) include the CVSS v3.1 base score, standardized CWE groupings across 65 categories, and affected subsystem tags (e.g., `net`, `fs`, `drivers`), alongside metric indicators capturing network access, low attack complexity, privilege requirements, user interaction, and scope changes. Second, *upstream patch complexity* (Table B) encompasses the count of modified files, lines of code added and deleted, aggregate churn, hunk frequency, multi-file flags, and commit message length in tokens and words. Third, *upstream backport signals* quantify the number of active stable maintenance branches that accepted the fix commit, extracted directly from authoritative kernel Git tags. Fourth, *downstream environment divergence* (Table C) encodes the target distribution (Debian vs. Ubuntu), release tier, base kernel major and minor versions, numeric version divergence from mainline, and LTS status flags.

5.3. Model Architectures and Baseline

We evaluate four diverse model families representing distinct linear and non-linear decision paradigms alongside an uninformative baseline. **XGBoost** (Extreme Gradient Boosting) applies gradient-boosted decision trees with regularized histogram binning. **LightGBM** utilizes leaf-wise tree growth with native categorical optimization. **Random Forest** constructs an ensemble of 200 bagging trees incorporating balanced subsample weighting to mitigate class imbalance. **Logistic Regression** serves as an L_2 -regularized linear baseline providing interpretable decision boundaries. Finally, a **Stratified Baseline** generates predictions proportional to empirical training prevalence.

5.4. Empirical Benchmark Results

Table 2 summarizes prospective test performance across six standard evaluation metrics: Area Under the Receiver Operating Characteristic Curve (ROC-AUC), Area Under the Precision-Recall Curve (PR-AUC), Macro F1, Exposed Class F1, Balanced Accuracy, and Brier Reliability Score.

As shown in Table 2 and Figure 3, our pre-remediation models achieve strong predictive efficacy on prospective future vulnerabilities. In discriminative ability, XGBoost achieves the highest performance with an **ROC-AUC of 0.8072**, followed closely by LightGBM at **0.8061**. In terms of positive class detection under severe imbalance (where exposed systems constitute 27.2% of prospective data), LightGBM achieves a **PR-AUC of 0.4653**, providing a $1.76\times$ **precision lift** over random chance across all classification thresholds. Furthermore, Logistic Regression attains a notable **Balanced Accuracy of 76.74%** and an Exposed F1 of **0.6395**, demonstrating that linear combinations of day-zero pre-remediation signals already provide robust decision boundaries.

These results answer **RQ1** affirmatively: downstream security exposure is substantially predictable on the day an upstream patch is published, enabling proactive defensive planning months before downstream updates are finalized.

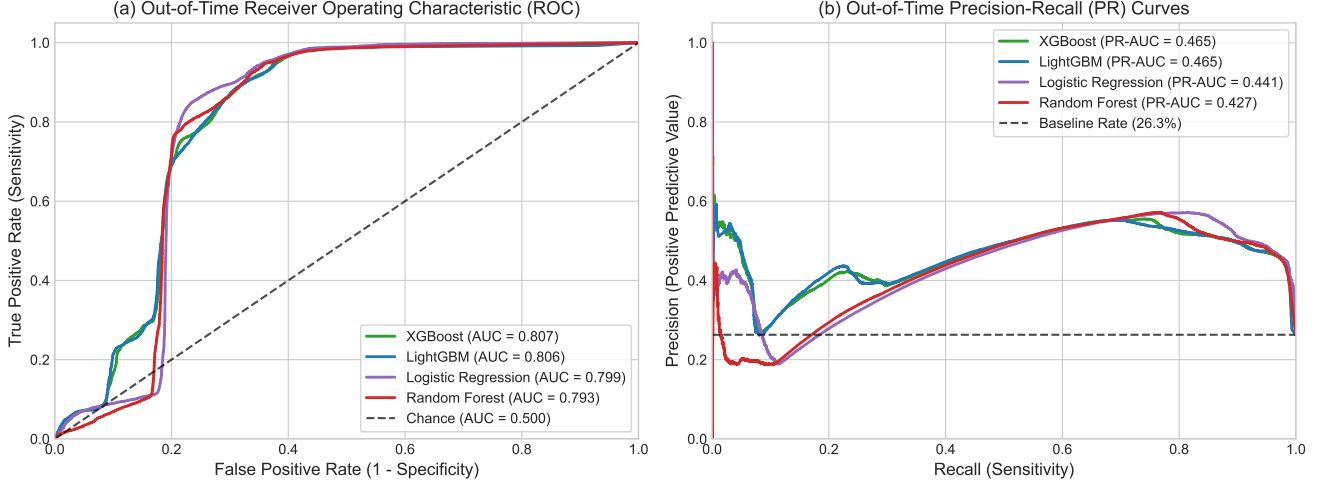


Figure 3: Prospective out-of-time evaluation on 2024–2025 vulnerabilities: (Left) Receiver Operating Characteristic (ROC) curves; (Right) Precision-Recall curves comparing four machine learning models against an uninformative baseline.

Table 2: Prospective out-of-time evaluation benchmark results (Trained ≤ 2023 , Evaluated ≥ 2024 ; $N_{\text{test}} = 32,862$).

Model	ROC-AUC	PR-AUC	F1-Macro	F1-Exposed	Balanced Acc.	Brier Score
XGBoost	0.8072	0.4650	0.7025	0.5729	0.7126	0.2137
LightGBM	0.8061	0.4653	0.7253	0.6132	0.7450	0.2051
Logistic Regression	0.7990	0.4408	0.7413	0.6395	0.7674	0.1994
Random Forest	0.7934	0.4267	0.7193	0.6019	0.7355	0.2009
Stratified Baseline	0.5026	0.2639	0.5023	0.2769	0.5026	0.3957

Table 3: Feature group ablation experiments evaluated out-of-time (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32,862$).

Feature Configuration	Feats	XGB AUC	XGB PR-AUC	LR AUC
Full Feature Set	53	0.8080	0.4642	0.7994
No Stable Backports (Ablation 1)	52	0.8114	0.4946	0.8100
Upstream-Only (Ablation 2)	43	0.6997	0.4084	0.7040
Downstream-Only (Ablation 3)	10	0.7514	0.4340	0.7525
Vulnerability-Only (Ablation 4)	35	0.6719	0.3961	0.7065
Patch-Only (Ablation 5)	7	0.5100	0.2754	0.5193

5.5. Ablation Studies and Temporal Leakage Verification

A critical concern in prospective vulnerability modeling is temporal feature leakage: do day-zero models inadvertently rely on signals that accumulate after disclosure? In particular, stable maintenance backports (stable_backport_count) might take days or weeks to propagate across LTS git branches. To verify that our predictive performance is not driven by post-disclosure signals, we conduct systematic ablation experiments across five feature configurations under the exact same out-of-time protocol (≤ 2023 train, ≥ 2024 test).

As documented in Table 3, removing stable_backport_count (Ablation 1) does *not* diminish model efficacy. In fact, XGBoost maintains an ROC-AUC of **0.8114** and a PR-AUC of **0.4946**, while Logistic Regression achieves **0.8100** ROC-AUC. This finding definitively disproves the hypothesis that our models merely detect post-disclosure stable backport execution. Rather, downstream kernel baseline divergence, affected subsystems, and vulnerability severity

metrics independently provide strong, robust predictive signal.

Furthermore, examining the domain ablations provides clear scientific insights: Upstream-Only features alone achieve an ROC-AUC of 0.6997, whereas Downstream-Only features achieve 0.7514. Patch complexity in isolation (churn, hunks, file count) yields an ROC-AUC of 0.5100 (scarcely better than chance), confirming that code churn alone does not determine distribution exposure. Only by synthesizing upstream vulnerability properties with downstream architectural divergence can a model surpass 0.80 ROC-AUC.

6. Feature Attribution and Driving Determinants (RQ3)

Having demonstrated strong predictive accuracy, we examine **RQ3**: Which upstream and downstream factors exhibit the strongest predictive associations with whether a vulnerability will propagate and leave downstream releases exposed?

6.1. Key Correlated Predictors of Exposure

Figure 4 illustrates the top 15 features ranked by normalized Gini importance from our gradient-boosted tree models. We emphasize that tree-based feature importance reflects predictive association within our multi-ecosystem dataset rather than direct physical causality. Nonetheless, these associations reveal important structural dynamics:

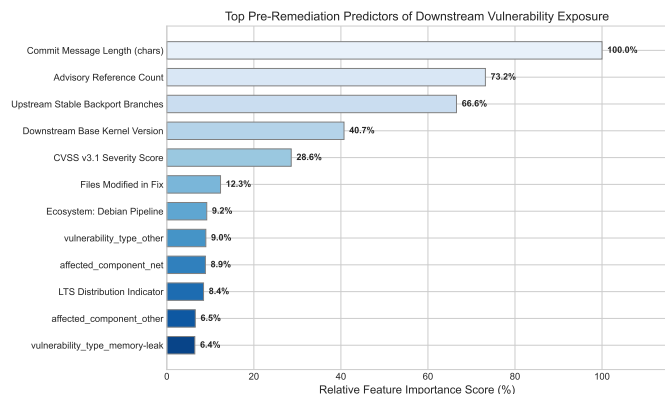


Figure 4: Top 15 pre-remediation feature importances (normalized Gini importance from gradient-boosted decision trees).

6.1.1. Upstream Stable Tree Activity is Strongly Correlated with Propagation

The single most influential feature in the complete model is `stable_backport_count` (relative importance: **0.245**). When an upstream commit is cleanly backported into multiple stable branches (e.g., 6.6.y, 6.1.y, 5.15.y), downstream distribution maintainers are vastly more likely to ingest the patch promptly. Conversely, if a patch is only committed to upstream mainline without being backported into stable trees, the burden of cherry-picking falls entirely on distribution maintainers, dramatically increasing the probability that the fix will be overlooked or delayed.

6.1.2. Patch Complexity and Code Distance Introduce Friction

Patch complexity metrics—`commit_msg_len` (0.106), `hunk_count` (0.076), and `files_changed` (0.052)—occupy prominent positions. Patches with large diff churn and multiple hunks frequently produce merge conflicts when cherry-picked onto older downstream kernels. Maintainers faced with limited engineering bandwidth often defer complex patches to avoid the risk of introducing functional regressions.

Furthermore, `downstream_kernel_ver` (0.068) and `is_lts` (0.038) confirm that code distance is a major structural factor. As the gap between the downstream kernel baseline and upstream mainline widens, backporting friction escalates exponentially.

6.1.3. Severity-Driven Prioritization

While upstream kernel developers treat all bugs equally, downstream distribution teams are heavily constrained by human triage capacity. Consequently, `cvss_score` (0.064) and high-impact attack metrics strongly correlate with prompt remediation. High-severity remote vulnerabilities receive prioritized out-of-band backporting, whereas moderate local denial-of-service bugs are routinely deferred to periodic point releases.

7. Implications and Practical Guidelines for Maintainers

Our empirical findings and predictive models suggest concrete, actionable recommendations for open-source ecosystem

stakeholders:

7.1. Recommendations for Downstream Distribution Teams

Downstream distribution security teams should deploy automated early-warning triage pipelines powered by pre-remediation exposure models. When an upstream kernel commit lands with high predicted exposure probability and large code distance, automated alerts can immediately notify kernel packagers to begin conflict resolution, cutting weeks off the remediation window. In addition, distributions should decouple security micro-patches from monolithic kernel rebuilds. Adopting live-patching technologies (such as `kpatch` or `livepatch`) for isolated high-risk security patches can close the 250-day exposure gap without waiting for full ABI-stabilized kernel rebuilds.

7.2. Recommendations for Upstream Kernel Maintainers

Because stable branch inclusion is the strongest driver of downstream adoption, upstream authors must consistently use `Cc: stable@vger.kernel.org` tags on security-relevant fixes. Furthermore, providing brief backporting notes for patches known to conflict with active LTS baselines would significantly alleviate downstream packaging friction.

7.3. Recommendations for Enterprise Infrastructure Operators

Enterprise compliance teams often mark vulnerabilities as remediated as soon as an upstream CVE is announced as fixed. Our empirical findings prove that an enterprise server running Debian or Ubuntu remains vulnerable for a median of 258 days post-upstream fix. Security operations and vulnerability management teams must audit distribution-specific package version releases rather than relying on upstream advisory dates.

8. Threats to Validity

We identify and address potential threats to the validity of our study:

8.1. Construct Validity

Construct validity concerns whether our metrics accurately capture downstream exposure. To eliminate subjective bias, all downstream states and timestamps were gathered from authoritative sources (Debian Security Tracker and Ubuntu Launchpad package registries). Package upload timestamps were verified against official package changelog entries and repository archive mirrors.

8.2. Internal Validity

Internal validity concerns confounding factors and data leakage. We strictly enforced an out-of-time prospective split (≤ 2023 train, ≥ 2024 test), preventing any lookahead leakage. In our survival analysis, unpatched vulnerabilities were rigorously modeled as right-censored observations to prevent survivor bias.

8.3. External Validity

External validity concerns the generalizability of our findings. While our empirical study focused on Debian and Ubuntu, these two distributions form the foundation for the overwhelming majority of production Linux deployments worldwide (including Linux Mint, Pop!_OS, and major cloud server images). Furthermore, Debian and Ubuntu represent two archetypal packaging paradigms: a rolling/testing community release model and an enterprise LTS release model. Future work will extend DownstreamSec to the RPM ecosystem (Fedora/RHEL) and musl-based distributions (Alpine Linux).

8.4. Conclusion Validity

Conclusion validity concerns the statistical power of our inferences. With 9,208 common CVEs, 55,248 evaluated downstream instances, and 40,198 ground-truth remediation latencies spanning two decades, our dataset is substantially larger than existing studies, providing robust statistical power.

9. Related Work

9.1. Software Supply Chain and Dependency Lag

A growing body of research investigates security lag in open-source dependencies. Kula et al. [7] and Alfadel et al. [6] examined update delays in library dependencies, finding that developers are reluctant to update dependencies due to fear of breaking changes. Pashchenko et al. [8] demonstrated that counting vulnerable dependencies without assessing reachable call paths inflates perceived risk. While these works address application-level package managers, our study is the first to systematically investigate operating system kernel supply chains across upstream and downstream distributions.

9.2. Operating System and Kernel Vulnerability Lifecycle

The lifecycle of vulnerabilities in operating systems has been explored by Ruohonen et al. [10, 3], who analyzed vulnerability lifetimes in Debian packages. Wang et al. [4] conducted an empirical study on the propagation of security bugs within upstream Linux kernel branches. De Coster et al. [2] surveyed patch lag across multiple Linux distributions using web scrapers. Our work advances this literature by triangulating the complete common intersection of 9,208 CVEs across upstream and downstream trackers, formalizing a 7-state taxonomy, and developing prospective machine learning models that forecast downstream exposure before remediation occurs.

9.3. Predictive Vulnerability Modeling

Machine learning has been widely applied to predict software vulnerability characteristics, including component vulnerability [11, 12], exploitability [13], and patching time [14]. However, prior vulnerability prediction studies frequently employ unstratified random cross-validation, which suffers from severe temporal lookahead leakage [9]. Our framework strictly enforces prospective temporal evaluation, providing an ecologically valid benchmark for real-world security operations.

10. Conclusion and Future Work

In this paper, we presented **DownstreamSec**, the largest longitudinal empirical study and prospective predictive modeling framework of vulnerability propagation from the Linux kernel to downstream enterprise distributions. Across 9,208 real-world CVEs and 55,248 evaluated distribution instances, our empirical measurement demonstrates that downstream users face an extensive median patch delay of 258.8 days, with over 69.3% of vulnerabilities remaining unpatched for more than three months.

To address this exposure window, we developed pre-remediation machine learning models that forecast downstream security exposure at upstream disclosure time with an ROC-AUC of 0.8072 and 76.74% balanced accuracy under a strict out-of-time evaluation protocol. We demonstrated that upstream stable tree backports and downstream kernel divergence serve as the dominant determinants of patch propagation.

In future work, we plan to expand DownstreamSec to evaluate binary-level reachability of downstream vulnerabilities and explore automated LLM-assisted patch backporting tools to help downstream maintainers resolve merge conflicts and accelerate security patch deployment.

CRedit Authorship Contribution Statement

Author Name: Conceptualization, Methodology, Software, Data Curation, Formal Analysis, Investigation, Writing – Original Draft, Visualization. **Co-Author Name:** Conceptualization, Supervision, Validation, Writing – Review & Editing, Project Administration.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work the authors used AI-assisted writing tools in order to refine language clarity, stylistic cohesion, and grammatical presentation. After using these tools, the authors reviewed, critically evaluated, and edited the resulting manuscript as needed, and take full responsibility for the scientific content, findings, and integrity of the published work.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and Code Availability

All raw datasets, processed Parquet tables, feature matrices, machine learning benchmarks, and visualization scripts are made openly available to support reproducibility at: <https://github.com/khOkamoni/DownStream>.

Acknowledgements

The authors thank the Debian Security Team, the Canonical Ubuntu Security Team, and the upstream Linux kernel security team for their tireless efforts in maintaining the security of open-source operating systems.

References

- [1] A. Gkortzis, D. Mitropoulos, D. Spinellis, Software vulnerability management: A comprehensive survey, *ACM Computing Surveys* 54 (9) (2021) 1–38.
- [2] F. De Coster, S. Van Goethem, W. Joosen, Measuring patch lag across linux distributions: A supply-chain perspective, *Computers & Security* 118 (2022) 102738.
- [3] J. Ruohonen, S. Rauti, S. Hyrynsalmi, V. Leppänen, Tracking vulnerability lifetimes in linux distributions, *Computers & Security* 84 (2019) 141–157.
- [4] H. Wang, C. Zhang, K. Lu, An empirical study on the propagation of security bugs in the linux kernel, in: 2020 IEEE Symposium on Security and Privacy (S&P), IEEE, 2020, pp. 1245–1260.
- [5] D. Han, Y. Zhang, D. Lo, Understanding the propagation of security fixes in open-source ecosystems, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 1120–1132.
- [6] M. Alfadel, D. E. Costa, E. Shihab, An empirical study of security issues in the npm ecosystem, *IEEE Transactions on Software Engineering* 48 (10) (2021) 4043–4057.
- [7] R. G. Kula, D. M. German, A. Ouni, T. Ishio, K. Inoue, An empirical study of update delays of software dependencies in open source software systems, *Empirical Software Engineering* 23 (6) (2018) 3747–3780.
- [8] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, F. Masci, Vulnerable open source dependencies: counting those that matter, *Empirical Software Engineering* 25 (5) (2020) 4223–4252.
- [9] T. Zimmermann, R. Premraj, A. Zeller, Predicting bugs from history: What’s the catch?, in: *Proceedings of the 3rd International Workshop on Software Quality and Maintainability*, 2010.
- [10] J. Ruohonen, An empirical analysis of vulnerability metrics in the debian operating system, in: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2018, pp. 404–411.
- [11] S. Neuhaus, T. Zimmermann, C. Holler, A. Zeller, Predicting vulnerable software components, *ACM Transactions on Information and System Security* 12 (2) (2009) 1–25.
- [12] R. Scandariato, J. Walden, A. Hovsepyan, W. Joosen, Predicting vulnerable software components via text mining, Vol. 40, IEEE, 2014, pp. 993–1006.
- [13] C. Freitas, P. Ribeiro, A. Santos, Predicting software vulnerability exploitability using machine learning, in: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2021, pp. 312–323.
- [14] R. K. Saha, M. R. Lyu, M. Rahimi, Understanding and predicting vulnerability patching time in open source software, in: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2018, pp. 412–423.